

SE LAB-3

NAME: PAVANKUMAR HS

SRN: PES2UG24CS344

SECTION: F

1. Functional Requirements

The system should allow:

1. Customers to **register and log in**.
2. Customers to **view available restaurants**.
3. Customers to **view available tables**.
4. Customers to **book a restaurant table**.
5. Customers to **view the restaurant menu**.
6. Customers to **pre-order food** before arriving.
7. Customers to **make payments** for their booking/pre-order.
8. Customers to **receive booking/order confirmation**.
9. Restaurant managers to **manage tables and availability**.
10. Restaurant managers to **manage menu items and pre-orders**.

2. Non-Functional Requirements

The system should provide:

- **Security** — protect customer and payment information.
- **Performance** — booking and ordering should respond quickly.
- **Usability** — the app should be simple and easy to navigate.
- **Reliability** — bookings should be stored correctly without losing data.
- **Scalability** — the system should support many customers and restaurants.
- **Availability** — users should be able to access the system reliably.

3. Main Challenges

Performance -The system must quickly show table availability and process bookings.

Security -Customer accounts and payment information must be protected.

Usability -Customers should be able to book tables and pre-order food easily.

Data consistency Two customers should not be able to successfully book the same table at the same time.

Step 2: Architectural Style Analysis

Your lab specifically asks you to analyze **3 architectural styles**:

A. Layered Architecture

Structure:

- Presentation Layer
- Business Logic Layer
- Data Layer

Pros:

- Clear separation of responsibilities.
- Easy to understand and maintain.
- Good for applications with organized business logic.

Cons:

- Communication between layers can add some overhead.
- Changes in one layer may sometimes affect other layers.
- Scaling individual parts can be difficult.

For our app: Suitable because the restaurant app has clear UI, booking/order logic, payment, and data management.

B. Microservices Architecture

Structure:

The application is divided into independent services such as:

- User Service
- Restaurant Service
- Table Booking Service
- Menu Service
- Payment Service
- Order Service

Pros:

- Individual services can be scaled independently.

- Services can be developed and deployed separately.
- Failure in one service can potentially be isolated.

Cons:

- More complex to develop and maintain.
- Communication between services can introduce network latency.
- Data consistency becomes more difficult.

For our app: Possible, but probably unnecessary for a college-level restaurant booking application.

C. Client-Server Architecture

Client/App



Server



Database

The customer uses the application as the client, while the server handles bookings, orders, payments, and restaurant information.

Pros:

- Centralized data management.
- Easy to control and manage data.
- Simple deployment compared with microservices.

Cons:

- Server can become a bottleneck.
- Server failure can affect the whole application.
- Scaling can be more difficult.

For our app: Suitable, but Layered Architecture gives better separation of responsibilities for our component modelling.

Architecture	Pros	Cons	Suitability
Layered	Simple, organized, easy to maintain	Some communication overhead	High
Microservices	Scalable, independent services	Complex, network latency	Medium
Client-Server	Centralized control, simple management	Server bottleneck, single point of failure	Medium

Step 3

Components:

1. User Interface Component
2. Order Manager Component
3. Restaurant & Table Management Component
4. Menu & Pre-Order Management Component
5. Payment Service Component

Component Responsibilities:

- User Interface: Allows customers to browse restaurants, book tables, view menus, and pre-order food.
- Order Manager: Manages table bookings and coordinates pre-orders and payments.
- Restaurant & Table Management: Manages restaurant information, table availability, and reservations.
- Menu & Pre-Order Management: Manages menu items, prices, and customer pre-orders.
- Payment Service: Processes payments for bookings and pre-orders.

STEP 4

Step 4: Component Diagram

Components

1. User Interface Component
2. Order Manager Component
3. Restaurant & Table Management Component
4. Menu & Pre-Order Management Component
5. Payment Service Component

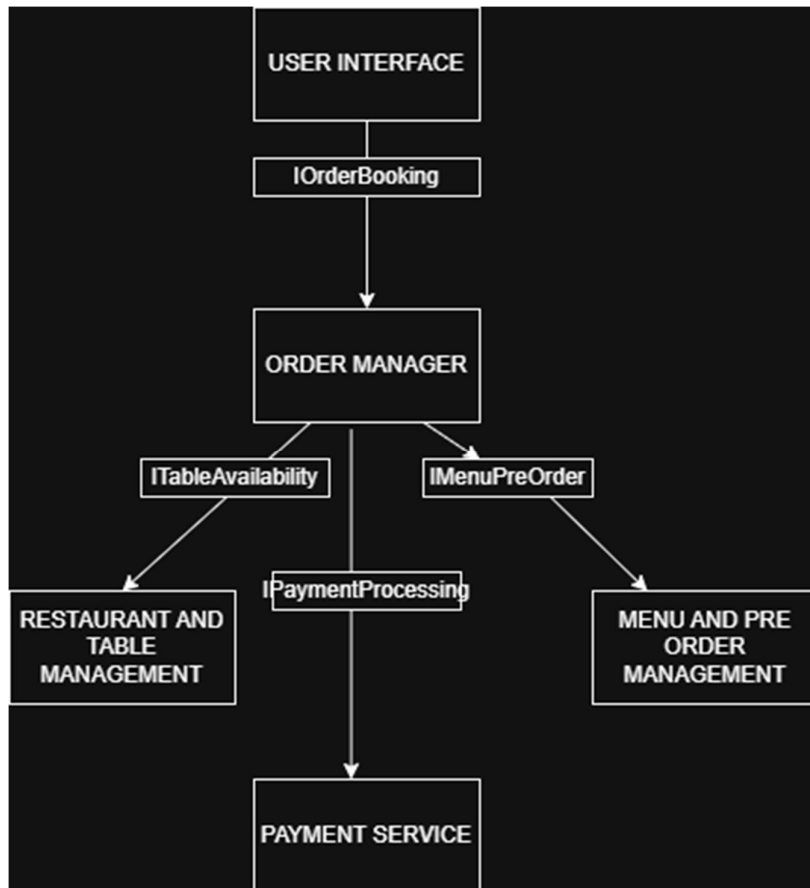
Interfaces

1. IOrderBooking — connects User Interface and Order Manager.
2. ITableAvailability — connects Order Manager and Restaurant & Table Management.
3. IMenuPreOrder — connects Order Manager and Menu & Pre-Order Management.
4. IPaymentProcessing — connects Order Manager and Payment Service.

Component Interactions

- User Interface sends table booking and pre-order requests to the Order Manager.
- Order Manager checks table availability through Restaurant & Table Management.

- Order Manager retrieves menu and manages pre-orders through Menu & Pre-Order Management.
- Order Manager sends payment requests to the Payment Service.



Architectural Style Selection & Justification

Architecture Selection

We chose Layered Architecture for the Restaurant Table Booking & Pre-Ordering App.

Architectural Choice

Layered Architecture separates the application into different layers/components such as the User Interface, Order Manager, Restaurant & Table Management, Menu & Pre-Order Management, and Payment Service. Each component has a specific responsibility and communicates through defined interfaces.

Two Reasons for the Choice

1. Clear separation of responsibilities:

The architecture separates user interaction, table booking, menu management, pre-ordering, and payment processing. This makes the system easier to understand, develop, test, and maintain.

2. Suitable for the application:

The Restaurant Table Booking & Pre-Ordering App has well-defined functions and does not require the complexity of independently deployed microservices. Layered Architecture provides a simple and organized structure for the system.

Security Advantage

The Payment Service is separated from the User Interface and Order Manager. This helps isolate payment-related operations and allows access to sensitive payment functionality to be controlled through defined interfaces.

Performance Benefit

The Order Manager can communicate directly with the required components to process bookings and pre-orders efficiently. This reduces unnecessary communication and helps provide faster responses to customers.

Conclusion

Layered Architecture is the most suitable choice because it provides clear separation of responsibilities, easier maintenance, improved security through component separation, and efficient processing for the restaurant booking and pre-ordering system.